



AI Lab Course Project Report



IUSemPlanner.ai v2.4 - Neural Academic Predictor

Submitted By:

Syed Imran Murtaza (65196)

Sahira Bibi (67859)

Muhammad Shehreyar (65193)

Course: Artificial Intelligence Lab

Instructor: Sir Rizwan Ahmed

Project Type: Web App + ML/ANN + Python FastAPI Backend

Live Website URL: <https://ctrlalTIMRan.com/IUSemPlanner/>

Project Files (GitHub Link):

<https://github.com/ctrlalTIMRan/IUSemPlanner.ai>

1. Project Overview

IUSemPlanner.ai is an academic planning and prediction system for Iqra University students. The main purpose is to help a student understand their academic situation from one place instead of checking different IULMS pages again and again.

The project imports data from IULMS, shows dashboard information, calculates GPA-related progress, checks attendance risk, displays timetable, supports course planning, and adds an ML/ANN Lab tab for prediction. The ML/ANN Lab predicts expected course grade, semester GPA, projected CGPA, confidence score, fail risk, and attendance debar risk.

In simple words: this project works like a smart academic assistant. It reads the student academic data, applies rules and machine learning, then gives useful warnings and predictions before the final result comes.

Item	Simple Explanation
Project name	IUSemPlanner.ai v2.4
Main purpose	Academic planning and grade prediction for students
Frontend	HTML, CSS, and Vanilla JavaScript
ML algorithm	Artificial Neural Network / Multi-Layer Perceptron
Python backend	FastAPI with scikit-learn MLPRegressor
Data source	Imported IULMS academic data for prediction input
Training data	Synthetic education-style data by default; real CSV training is supported
Storage	Browser LocalStorage; optional Supabase cloud sync

2. Problem Statement

Students usually get their academic information from separate IULMS sections. Attendance is in one place, transcript is in another place, midterm marks are shown separately, and timetable is also separate. Because of this, many students cannot easily understand their complete academic position.

The major problem is that students often know about risk very late. For example, a student may not realize that attendance is near the debar limit, or that current midterm marks may affect SGPA and CGPA. Manual calculation is also time-consuming and can create mistakes.

Therefore, the problem statement of this project is:

To develop an AI-based academic planning system that imports student data from IULMS, predicts future academic performance using ML/ANN, explains the reason behind risk, and helps students take better study and attendance decisions.

3. Aim and Objectives

The aim of the project is to make student academic planning easier, smarter, and more practical using artificial intelligence and machine learning.

- Import academic data from IULMS using a bookmarklet-based approach.
- Show transcript, attendance, timetable, courses, midterms, and exam schedule in one dashboard.
- Predict expected grade points for current courses using an ANN model.
- Calculate predicted SGPA and projected CGPA from course predictions.
- Detect attendance/debar risk using hard university attendance rules.
- Show confidence level using an ensemble of neural networks.
- Explain prediction using feature contribution / occlusion sensitivity.
- Provide a what-if simulator so students can change attendance or marks and see the prediction update.
- Provide optional Python FastAPI backend for ML prediction service.

4. Why this Project is AI/ML Based

This project is not only a normal dashboard because it does not just display data. It also learns patterns from training data and predicts a future value: expected grade points. This makes it an ML-based project.

The ANN takes input features such as midterm score, quizzes, project marks, attendance, previous CGPA, credit hours, lab flag, and course level. From these values, it predicts grade points between 0 and 4. This is a regression problem because the output is a numeric value.

AI/ML Concept	Where it is used in this project
Regression	Predicting grade points from 0 to 4
Artificial Neural Network	Learning non-linear relationship between marks, attendance, CGPA, and result
Deep Ensemble	Training 5 networks and taking average prediction
Uncertainty / Confidence	Using spread of ensemble predictions to estimate confidence
Feature Engineering	Converting raw IULMS data into 8 numeric ML features
Explainable AI	Showing which feature increased or decreased the prediction

5. System Modules

Module	Purpose
IULMS Import Module	Imports profile, course list, transcript, attendance, schedule, midterms, and exam schedule.
Parser Module	Converts raw IULMS text/JSON into clean JavaScript objects.
Dashboard Module	Shows CGPA, attendance health, course progress, and summary cards.

Timetable Module	Builds weekly class timetable from imported schedule.
Courses / Library Module	Shows course information, topics, and study guidance.
Planner Module	Helps plan courses and avoid academic issues.
AI Insights Module	Uses AI provider abstraction for academic advice and OCR/screenshot text extraction.
ML/ANN Lab Module	Trains ANN, predicts grades, explains risks, and runs what-if simulation.
Python Backend Module	Optional FastAPI service for server-side ANN predictions.

6. User Flow Diagram

The user flow is simple. First the student opens the website, then imports IULMS data. After import, the dashboard displays academic information and the ML/ANN Lab can run predictions.

User Flow Diagram - IUSemPlanner.ai ML/ANN Lab

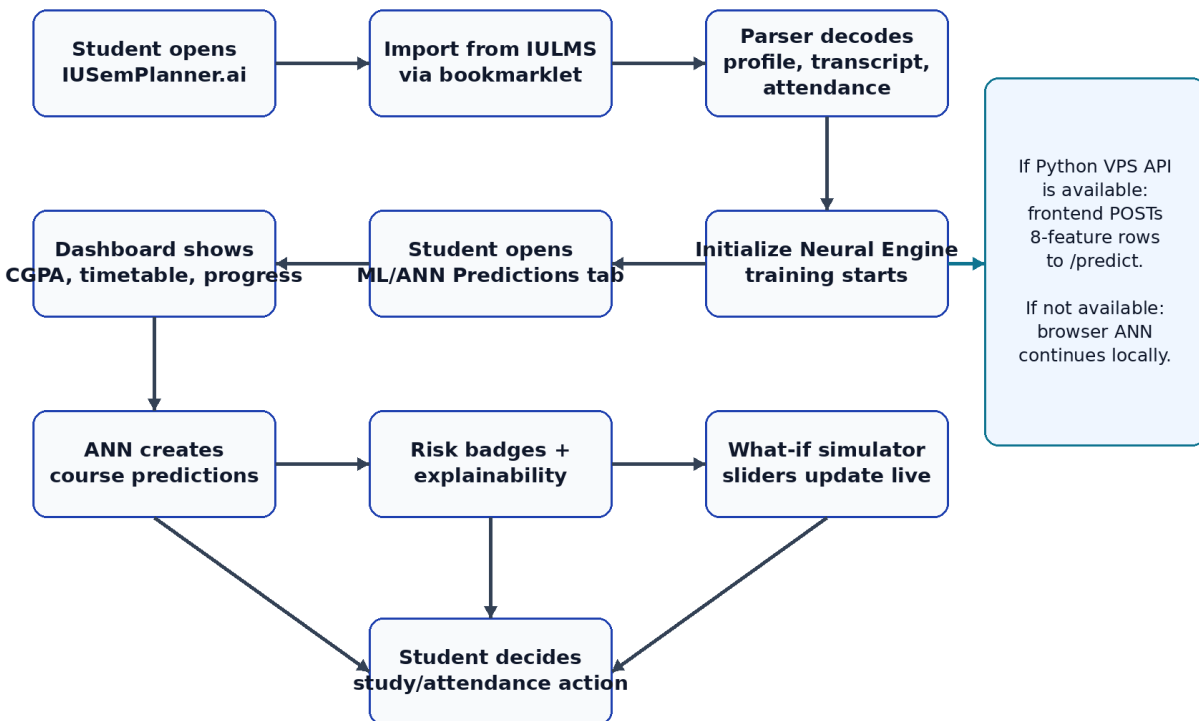


Figure 1: User flow of IUSemPlanner.ai

Easy viva explanation: The student imports data, the system cleans it, the dashboard shows it, and the ANN predicts academic outcome from the cleaned features.

7. System Architecture

The project has two main sides: the browser frontend and the optional Python ML backend. The browser side can work alone because the ANN is also implemented in JavaScript. If the Python backend URL is configured, the frontend can send features to FastAPI and receive predictions from the trained scikit-learn model.

System Architecture

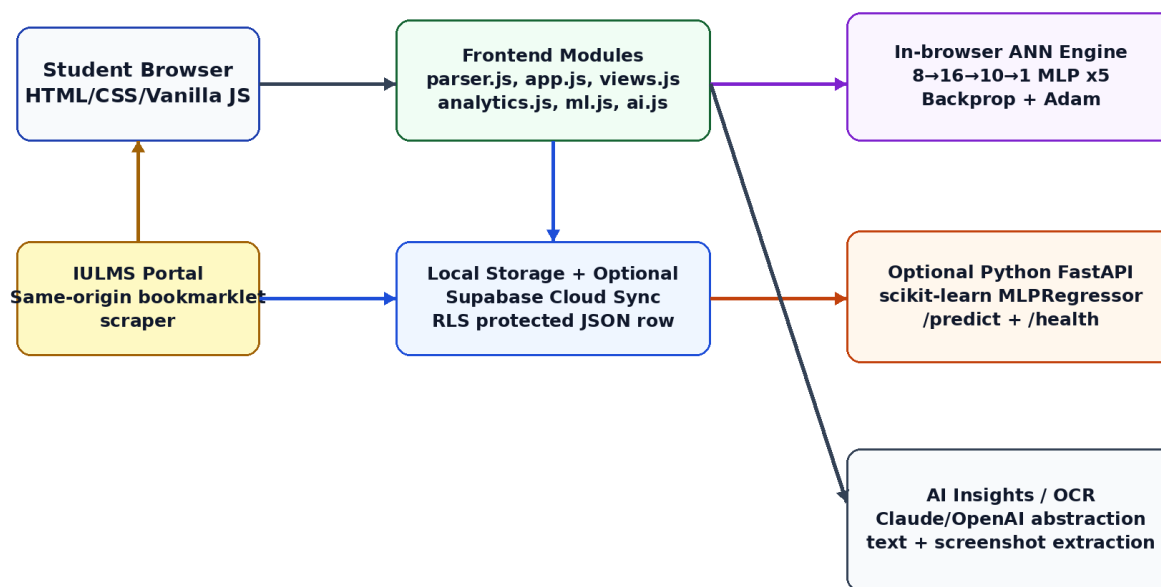


Figure 2: Overall architecture of the system

Layer	Technologies / Files	Role
Presentation Layer	index.html, styles.css, css/ios.css	User interface and responsive layout
Application Layer	app.js, views.js, parser.js, analytics.js	State handling, rendering, parsing, calculations
ML Layer in Browser	js/ml.js	ANN training, prediction, confidence, explanations
Backend ML Layer	ml-backend/app.py, train.py	FastAPI prediction API and scikit-learn training
Storage Layer	LocalStorage, optional Supabase	Save user academic data locally or in cloud
AI Layer	ai.js	AI advisor and OCR/text extraction abstraction

8. Dataset and Input Features

The model uses 8 input features. These features are selected because they directly affect student academic performance. All features are normalized between 0 and 1 so the neural network can train smoothly.

No.	Feature	Range	Meaning
1	Midterm score	0 to 1	Obtained midterm marks divided by total midterm marks
2	Quizzes / assignments	0 to 1	Continuous assessment performance
3	Project marks	0 to 1	Project or practical work score
4	Attendance	0 to 1	Present sessions divided by total sessions
5	Prior CGPA	0 to 1	Previous CGPA divided by 4
6	Credit hours	0 to 1	Course credit weight divided by 4
7	Lab flag	0 or 1	1 if the course is lab/practical, otherwise 0
8	Course level	0 to 1	Course difficulty level such as 100, 200, 300, or 400 level

Important honesty point: By default, the ANN is trained on synthetic education-style data. This means we did not claim that real university records were used for training. Real imported IULMS data is used as input for each student prediction. The backend also supports real CSV training if historical data becomes available.

9. Algorithms Used

The project uses the following main ML/ANN algorithms and supporting techniques. The algorithm names remain technical, but their working is explained in easy words.

Algorithm / Technique	Used For	Simple Working
Multi-Layer Perceptron (MLP)	Main prediction model	Takes 8 features and predicts grade points using connected neuron layers.
Backpropagation	Training the ANN	Calculates prediction error and sends correction backward through the network.
Adam Optimizer	Updating weights	Improves gradient descent by using adaptive learning rates.
Deep Ensemble	Confidence and stable prediction	Trains 5 networks and averages their outputs.
StandardScaler / Normalization	Preprocessing	Keeps input features on similar scale for better training.
Bootstrap Sampling	Diversity in ensemble	Each network trains on a slightly different sample of training data.
Occlusion Sensitivity	Explainability	Replaces one feature with average value and checks how prediction changes.
Rule-based Layer	Attendance debar risk	Applies known university rule separately from ML model.

Feature Selection:

FEATURES = [

```
"midterm",  
"quizzes",  
"project",  
"attendance",  
"cgpa",  
"credits",  
"isLab",  
"level"  
]
```

Synthetic Dataset Generation:

```
def clamp01(x):  
    return np.clip(x, 0, 1)  
  
def generate_synthetic_data(n=1400):  
    rng = np.random.default_rng(42)  
  
    ability = clamp01(0.60 + 0.16 * rng.standard_normal(n))  
    cgpa = clamp01(ability + 0.08 * rng.standard_normal(n))  
  
    attendance = clamp01(  
        0.55 + 0.45 * clamp01(ability + 0.25 * rng.standard_normal(n))  
    )  
  
    midterm = clamp01(  
        0.72 * ability + 0.18 * attendance + 0.12 * rng.standard_normal(n)  
    )  
  
    quizzes = clamp01(  
        0.65 * ability + 0.20 * attendance + 0.15 * rng.standard_normal(n)  
    )  
  
    project = clamp01(  
        0.60 * ability + 0.40 * rng.random(n)  
    )  
  
    level = rng.choice([0.25, 0.50, 0.75, 1.00], size=n)  
    is_lab = (rng.random(n) < 0.25).astype(float)  
  
    credits = np.where(is_lab > 0, 0.25, 0.75)  
  
    grade_point = 4 * clamp01(  
        0.40 * midterm  
        + 0.12 * quizzes  
        + 0.06 * project  
        + 0.20 * attendance
```

```

    + 0.16 * cgpa
    + 0.06 * is_lab
    - 0.10 * (level - 0.25)
    + 0.07 * rng.standard_normal(n)
)

grade_point = np.clip(grade_point, 0, 4)

X = np.column_stack([
    midterm,
    quizzes,
    project,
    attendance,
    cgpa,
    credits,
    is_lab,
    level
])

y = grade_point

return X, y

```

Date Preprocessing:

```

X, y = generate_synthetic_data()

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

ANN Model Creation

```

ann_model = MLPRegressor(
    hidden_layer_sizes=(16, 10),
    activation="tanh",
    solver="adam",
    learning_rate_init=0.01,
    max_iter=600,

```



```
random_state=42  
)
```

ANN Model Training

```
ann_model.fit(X_train_scaled, y_train)
```

Model Prediction

```
y_pred = ann_model.predict(X_test_scaled)  
  
y_pred = np.clip(y_pred, 0, 4)
```

ANN Code

```
import numpy as np  
import joblib  
  
from sklearn.neural_network import MLPRegressor  
from sklearn.preprocessing import StandardScaler  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import mean_absolute_error  
  
def clamp01(x):  
    return np.clip(x, 0, 1)  
  
def generate_synthetic_data(n=1400):  
    rng = np.random.default_rng(42)  
  
    ability = clamp01(0.60 + 0.16 * rng.standard_normal(n))  
    cgpa = clamp01(ability + 0.08 * rng.standard_normal(n))  
  
    attendance = clamp01(  
        0.55 + 0.45 * clamp01(ability + 0.25 * rng.standard_normal(n))  
    )  
  
    midterm = clamp01(  
        0.72 * ability + 0.18 * attendance + 0.12 * rng.standard_normal(n)  
    )  
  
    quizzes = clamp01(  
        0.65 * ability + 0.20 * attendance + 0.15 * rng.standard_normal(n)  
    )
```

```

project = clamp01(
    0.60 * ability + 0.40 * rng.random(n)
)

level = rng.choice([0.25, 0.50, 0.75, 1.00], size=n)
is_lab = (rng.random(n) < 0.25).astype(float)

credits = np.where(is_lab > 0, 0.25, 0.75)

grade_point = 4 * clamp01(
    0.40 * midterm
    + 0.12 * quizzes
    + 0.06 * project
    + 0.20 * attendance
    + 0.16 * cgpa
    + 0.06 * is_lab
    - 0.10 * (level - 0.25)
    + 0.07 * rng.standard_normal(n)
)

grade_point = np.clip(grade_point, 0, 4)

X = np.column_stack([
    midterm,
    quizzes,
    project,
    attendance,
    cgpa,
    credits,
    is_lab,
    level
])

y = grade_point

return X, y

X, y = generate_synthetic_data()

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

scaler = StandardScaler()

```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

ann_model = MLPRegressor(
    hidden_layer_sizes=(16, 10),
    activation="tanh",
    solver="adam",
    learning_rate_init=0.01,
    max_iter=600,
    random_state=42
)

ann_model.fit(X_train_scaled, y_train)

y_pred = ann_model.predict(X_test_scaled)
y_pred = np.clip(y_pred, 0, 4)

mae = mean_absolute_error(y_test, y_pred)

print("Mean Absolute Error:", mae)

sample_student = np.array([[
    0.75,
    0.80,
    0.70,
    0.88,
    0.82,
    0.75,
    0,
    0.50
]])

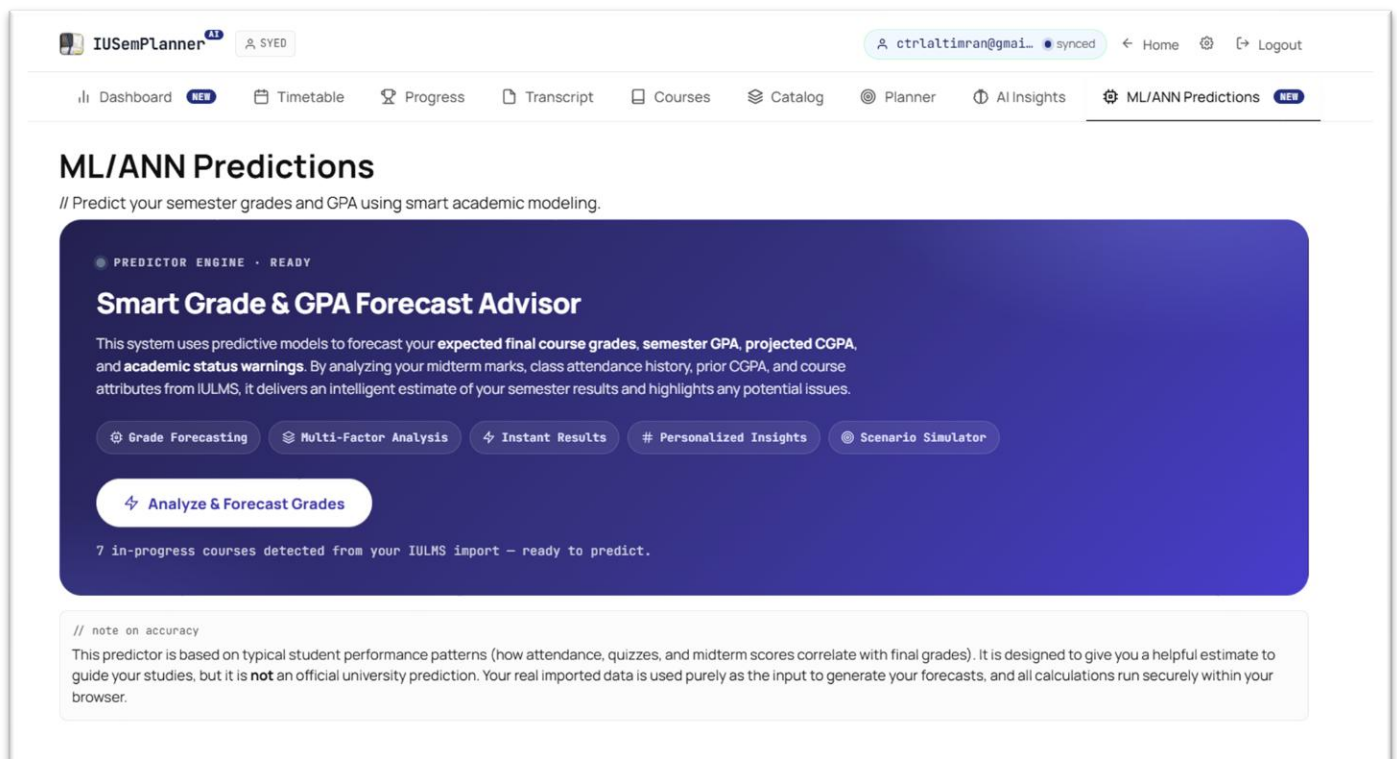
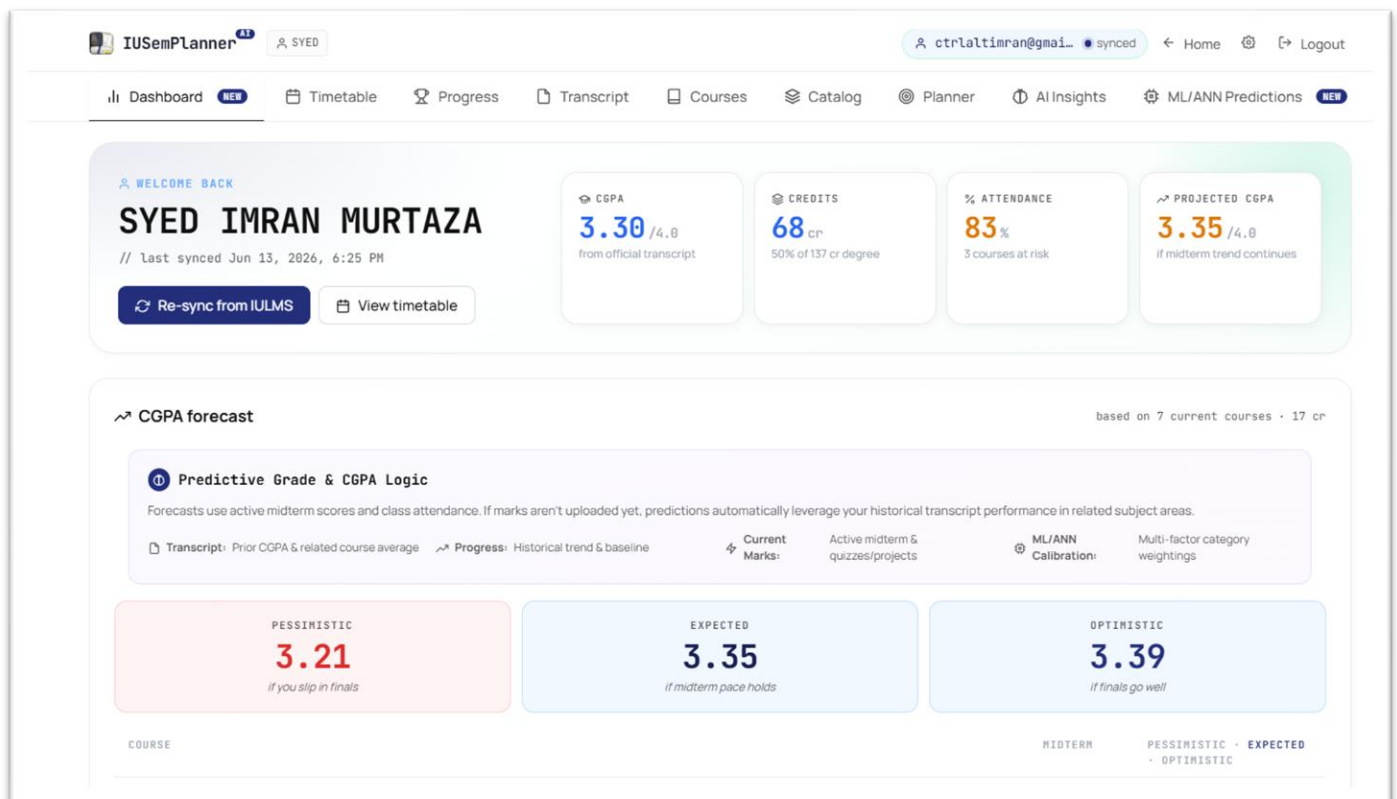
sample_student_scaled = scaler.transform(sample_student)

predicted_gp = ann_model.predict(sample_student_scaled)
predicted_gp = np.clip(predicted_gp, 0, 4)

print("Predicted Grade Point:", predicted_gp[0])

joblib.dump(ann_model, "ann_grade_predictor.pkl")
joblib.dump(scaler, "scaler.pkl")
```

Screenshots:



IUSemPlanner^{AI}

SYED

ctrlaltrimran@gmail...synced

Home

Logout

Dashboard

Timetable

Progress

Transcript

Courses

Catalog

Planner

AI Insights

ML/ANN Predictions

ML/ANN Predictions

// Predictions generated from your imported IULMS data · Model Accuracy: ±0.22 Grade Points

Retrain

PREDICTED SEMESTER GPA

2.77

Estimated range: ±0.10 GP

PROJECTED CGPA

3.19

current 3.30 → estimated after this sem

COURSES AT RISK

3

0 on watch · 4 on track

METHOD

Deep Ensemble

Analyzes 8 performance factors

ANN Prediction Logic & Deep Ensemble Calibration

Our Artificial Neural Network maps current midterm marks, attendance, and historical subject performance to predict final outcomes. Missing marks are dynamically imputed using similar course groups.

Transcript:

Blends historical subject GPA with network outputs

Progress:

Uses credit load & overall CGPA baselines

Current Marks:

Active midterm & quizzes/projects

Ensemble Spread:

Variance across 5 neural networks

SEN356

INFORMATION SECURITY

ON TRACK

B

Predicted: 2.77 GP

Expected: C+ to B

COURSE COMPONENTS

Midterm (20m): 15/20

Assessments (40m): Not uploaded yet

Final Exam (40m): Not uploaded yet

SEN355

WEB ENGINEERING

DEBAR RISK

C+

Predicted: 2.59 GP

Expected: C to B

COURSE COMPONENTS

Midterm (20m): 20/20

Assessments (40m): Not uploaded yet

Final Exam (40m): Not uploaded yet

SEN355-L

WEB ENGINEERING (LAB)

ON TRACK

C+

Predicted: 2.67 GP

Expected: C+ to B

COURSE COMPONENTS

Midterm (20m): 12/20

Assessments (40m): Not uploaded yet

Final Exam (40m): Not uploaded yet

IUSemPlanner^{AI}

SYED

ctrlaltrimran@gmail...HomeLogout

3

Live analysis

updates as you add/edit courses

1 clash — must resolve before registering

MON: SEN356 → SEN355 (1h 30m)

BALANCE SCORE

71 /100

good

CREDITS PLANNED

6 cr

2 sessions · 3h/week

AFTER THIS SEM

54 %

74/137 cr complete

CONFLICTS

1

must fix

DEAD HOURS

0m

0 gaps of 1h+

HEAVIEST DAY

MON

2 sess · 3h

FREE DAYS

6

TUE, WED, THU, FRI, SAT, SUN

DIFFICULTY MIX

100 /100

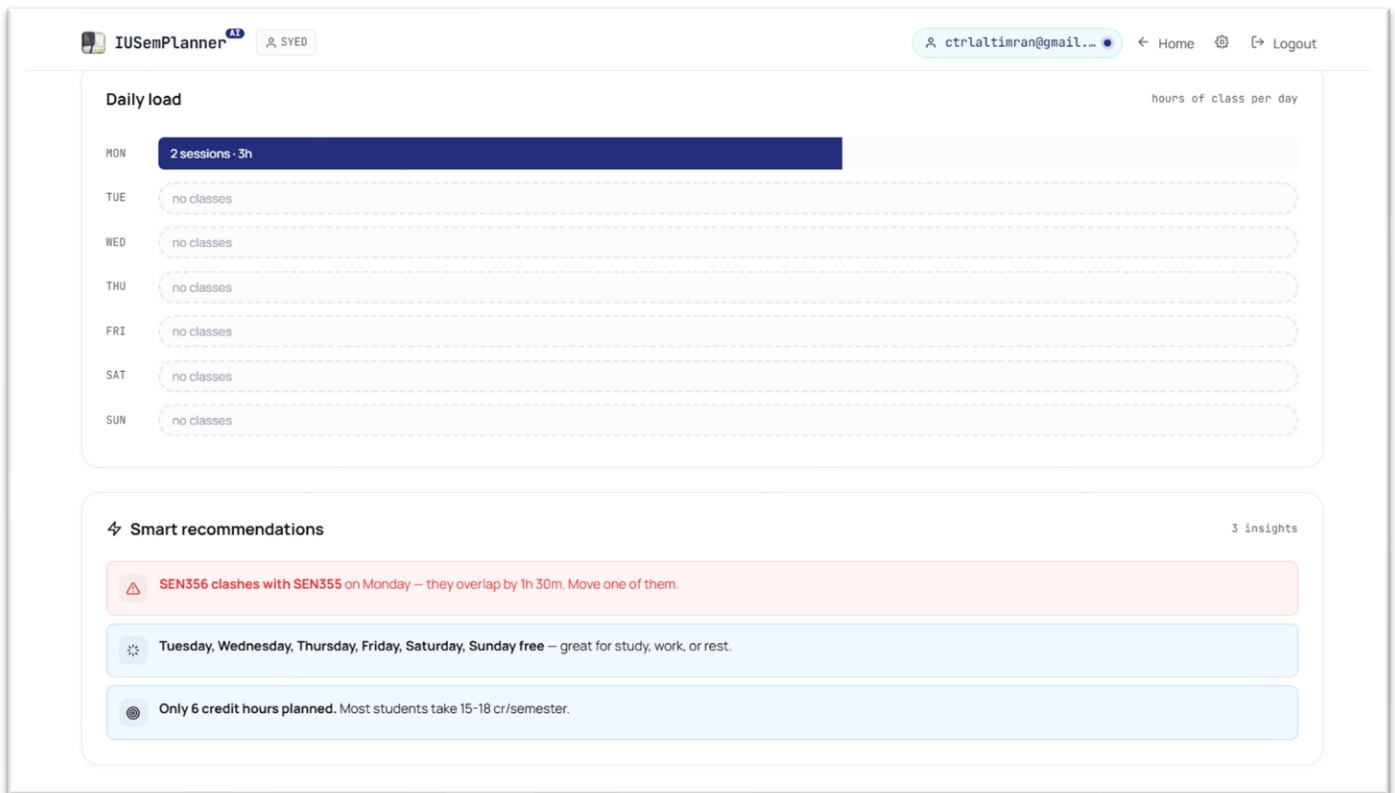
mental load spread

Weekly schedule preview

8 AM - 6 PM

time	MON	TUE	WED	THU	FRI
SAT	SUN				
8AM					
9AM	SEN355				

IUSemPlanner.ai - AI Lab Viva Friendly Report



11. Why These Algorithms Were Chosen

Choice	Reason
MLP instead of CNN	CNNs are best for images, but our data is tabular numeric data.
MLP instead of RNN	RNNs are for sequence/time-series text-like data, but our input is 8 fixed features.
MLP instead of Transformer	Transformer is too heavy and unnecessary for this small tabular regression problem.
MLP instead of simple linear regression	Student performance is not fully linear. For example, low attendance can suddenly create debar risk.
Deep ensemble	One model can be overconfident. Five models give more reliable average and confidence.
Rule-based attendance layer	University attendance rule is fixed, so it should be directly applied, not only learned.
Occlusion sensitivity	It is simple to explain in viva and shows which input affected the result.

Viva line: We chose MLP because our problem is a small tabular regression problem with non-linear relationships. The model is powerful enough for prediction but still simple enough to explain and implement.

12. Python ML Backend

The Python backend is optional but important for the AI Lab course because it shows a proper ML service using FastAPI and scikit-learn. The backend trains the same type of ANN using MLPRegressor and exposes two API endpoints.

Endpoint	Purpose
GET /health	Checks whether the ML API is running and returns model information.
POST /predict	Receives feature rows and returns predicted grade points and uncertainty.

The training file train.py generates or loads data, scales features, trains 5 MLPRegressor models, calculates validation MAE, and saves the model files using joblib. The app.py file loads the saved models and serves predictions through FastAPI.

The validation metric used is Mean Absolute Error (MAE). In simple words, MAE tells the average prediction error in grade points. If MAE is 0.22, it means predictions are off by around 0.22 grade points on average on the validation data.

13. Frontend ML/ANN Engine

The frontend also includes a complete ANN implementation in JavaScript inside js/ml.js. This is important because the project can work even without a server. The browser ANN trains live when the user opens the ML/ANN Lab and clicks Initialize Neural Engine.

- It implements dataset generation, feature normalization, MLP network creation, forward pass, backpropagation, Adam optimizer, ensemble training, prediction, confidence, and explainability.
- If the Python API is available, the frontend can use backend predictions.
- If the Python API is not available, the frontend automatically falls back to the browser ANN.
- This makes the project reliable for static hosting and also suitable for VPS ML deployment.

14. Prediction Output and Explainability

The project does not only show a number. It also explains risk in a way students can understand.

Output	Meaning
Expected grade	Predicted grade letter and grade points for each current course.
Range	Pessimistic and optimistic estimate based on uncertainty.
SGPA	Predicted semester GPA using credit-weighted average.
Projected CGPA	Current CGPA combined with expected semester performance.
Risk badge	ON TRACK, WATCH, or HIGH RISK.
Fail probability	Risk estimate from ensemble prediction spread.
Attendance debar warning	Separate hard rule check for attendance below threshold.
Confidence score	Higher confidence when ensemble outputs are close and inputs are complete.
Explanation	Shows which features helped or hurt the predicted result.

Occlusion sensitivity is used for explanation. It works like this: the model predicts once normally, then one feature is replaced with its average value. If the prediction changes a lot, that feature was important. This gives a simple explanation such as attendance lowered prediction or midterm improved prediction.

15. Testing and Evaluation

The system was tested at both application level and ML level. The goal was to check that data import, dashboard rendering, predictions, and fallback behavior work properly.

Test Case	Expected Result
Open app without login	Guest mode should work with LocalStorage.
Import IULMS data	Data should parse into profile, transcript, attendance, schedule, and midterms.
Initialize ML engine	Training should start and progress should be shown.
Predict courses	Each current course should show predicted grade and risk.
Low attendance course	Debar / high-risk warning should appear.
What-if sliders	Changing attendance or marks should update prediction live.
FastAPI /health	Backend should return ok, model architecture, and MAE.
FastAPI /predict	Backend should return gp and std for each feature row.
Backend unavailable	Frontend should fall back to browser ANN and still work.

For ML evaluation, the backend splits the dataset into training and validation data. The ensemble prediction is compared with validation target values using MAE. The reported MAE is approximate because default training uses synthetic data.

16. Limitations

- The default training data is synthetic, not real university historical result data.
- Prediction accuracy can improve if real past student performance records are available.
- Some IULMS pages may change their structure, so parser updates may be needed in future.
- AI advisor quality depends on the configured AI provider and available data.
- The system predicts academic risk but does not guarantee the final result because final exams and teacher marking can vary.

17. Future Enhancements

- Train the ML model on real anonymized historical student data after permission.
- Add more ML models such as Random Forest or XGBoost for comparison.
- Add model performance comparison chart in the report/dashboard.
- Add student-specific study recommendations based on weak courses.
- Add automatic exam preparation planner based on predicted risk.
- Improve OCR extraction for screenshots and PDFs.
- Deploy Python backend with HTTPS domain and restricted CORS.

18. Conclusion

IUSemPlanner.ai is a complete AI Lab project because it combines a real student problem with data import, dashboard analytics, machine learning prediction, neural network implementation, explainability, and an optional Python API backend. The project uses an MLP neural network with backpropagation, Adam optimizer, deep ensemble confidence, and rule-based attendance checking.

The most important point is that the project is practical. It does not use AI only for naming. It actually converts academic data into predictions and warnings that can help students make better decisions before exams.